Building a specialized search engine for attorneys requires a **Retrieval-Augmented Generation (RAG)** architecture. Because LLMs hallucinate—a fatal flaw in legal practice—you cannot rely on the model's internal memory. Instead, you must build a system where the LLM only reads and synthesizes case law that your system explicitly retrieves from a trusted local database.

Here is the architectural blueprint for your system, followed by the script to build your local repository.

System Architecture

To achieve absolute grounding in US case law, your architecture must be split into two distinct pipelines: **Data Ingestion** (building the brain) and **Search & Retrieval** (answering the queries).

1. The Tech Stack

- **Frontend:** Next.js (React) or Vue for the attorney-facing chat and search interface.
- **Backend/Orchestration:** Python with FastAPI, utilizing LangChain or LlamaIndex to manage the LLM routing and vector database querying.
- **Vector Database (The Repository):** PostgreSQL with pgvector or a dedicated system like Milvus or Qdrant. This stores the case text alongside its mathematical representation (embeddings).
- **Embedding Model:** Allow the user to select from a settings menu text-embedding-3-small (OpenAI), Claude's text embeddings or Ollama's text embedding to convert case text into vectors.
- **LLM Engine:** Allow the user to select from a settings menu GPT-4o, Claude 3.5 Sonnet, or Ollama. Make claude the default. Have the settings menu have the ability to input the api key for claude or chatgpt.

2. The Search & Retrieval Flow

When an attorney types a query into your website, the system executes this strict sequence:

1.Vectorize the Query:
The attorney enters a natural language search (e.g., "Texas cases on implied warranty of habitability in commercial leases"). The backend uses an embedding model to convert this text into a dense mathematical vector.

2.Semantic Search:
The system queries your Vector Database, comparing the query's vector against millions of pre-computed case law vectors. It retrieves the top 10 to 20 most relevant case chunks (paragraphs of opinions).

3.Strict Prompt Assembly:
The backend builds a prompt for the LLM that forces grounding. It injects the retrieved case text and instructs: *"You are an expert attorney. Answer the user's query using ONLY the following case law text. You must cite the specific case name and reporter for every claim. If the answer is not in the text, state 'I cannot answer this based on the retrieved cases.'"*

4.Generation:
The LLM processes the injected case text and generates the final, hallucination-free response for the attorney, complete with localized citations.

To see exactly how the data moves between these components during a search, explore this architectural flow:

Building the Grounded Repository

To ground your LLM in actual US case law, you need the raw text. The standard source for this is the **Free Law Project (CourtListener)**, which provides open-access bulk downloads of over 10 million legal opinions.
Because these files are massive (often hundreds of gigabytes uncompressed), your import script cannot load them entirely into memory. The script below safely streams the bulk download to your local machine and extracts it.

Python Import Script

You will need the requests library (pip install requests).

```python
import os
import requests
import tarfile

# CourtListener provides bulk data files. You will need to register a free account
# on courtlistener.com to generate your specific bulk download URL for "Opinions".
# Replace this URL with your generated bulk download link.
BULK_DATA_URL = "https://www.courtlistener.com/api/bulk-data/opinions/latest.tar.gz"
DOWNLOAD_PATH = "./courtlistener_opinions.tar.gz"
EXTRACT_DIR = "./case_law_repository"

def download_bulk_data(url, dest_path):
    print(f"Starting download from {url}...")

    # Use streaming to prevent memory overload on massive files
    with requests.get(url, stream=True) as response:
        response.raise_for_status()

        # Get total file size for tracking progress
        total_size = int(response.headers.get('content-length', 0))
        downloaded = 0

        with open(dest_path, 'wb') as file:
            for chunk in response.iter_content(chunk_size=8192):
                if chunk:
                    file.write(chunk)
                    downloaded += len(chunk)

                    # Print progress every ~50MB to avoid console spam
                    if downloaded % (50 * 1024 * 1024) < 8192:
                        print(f"Downloaded: {downloaded / (1024*1024):.2f} MB of {total_size / (1024*1024):.2f} MB")

    print("Download complete.")

def extract_archive(archive_path, extract_to):
    print(f"Extracting {archive_path} to {extract_to}...")

    os.makedirs(extract_to, exist_ok=True)

    # Extract the tar.gz file
    with tarfile.open(archive_path, "r:gz") as tar:
```

```
        tar.extractall(path=extract_to)

    print("Extraction complete. Your repository is ready for processing.")

if __name__ == "__main__":
    try:
        download_bulk_data(BULK_DATA_URL, DOWNLOAD_PATH)
        extract_archive(DOWNLOAD_PATH, EXTRACT_DIR)

        # Optional: Clean up the compressed file to save disk space
        os.remove(DOWNLOAD_PATH)
        print("Cleaned up compressed archive.")

    except Exception as e:
        print(f"An error occurred: {e}")
```

Next Steps for the Data

Once you have the raw case files downloaded, you cannot simply hand them to the Vector Database. You must write a secondary processor that:

1. **Cleans:** Strips HTML/XML tags from the CourtListener text files.
2. **Chunks:** Splits long 50-page opinions into smaller 500-word blocks (so the LLM doesn't get overwhelmed by too much text).
3. **Embeds:** Passes those chunks through your embedding model.
4. **Upserts:** Loads the resulting vectors and metadata (Case Title, Date, Court) into your Vector Database.